

# Capítulo 17

Multimodal

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 10/08/2026

# Capítulo 17 — Multimodal

**Objetivo de aprendizagem:** fazer o **mesmo Transformer** dos capítulos anteriores receber imagens. Construir um VQ-VAE do zero para transformar pixels em **tokens discretos**, e então descobrir que o modelo de texto funciona sem nenhuma alteração.

**Pré-requisitos:** Capítulos 5 (Transformer), 6 (tokenização) e 11 (pipeline de dados).

**Arquivos:**

- `preparar_dados.py` — baixa o MNIST
- `vqvae.py` — o VQ-VAE do zero: encoder, codebook, decoder
- `gerar_imagens.py` — o GPT dos capítulos de texto, gerando imagens
- `exercicios.md` — exercícios

## 1. Por que um Transformer não come imagens

Um Transformer recebe **tokens**: inteiros de um vocabulário finito. Uma imagem é o oposto — números contínuos, e muitos deles.

O MNIST é pequeno e já mostra o problema:

| Grandeza                      | Valor             |
|-------------------------------|-------------------|
| pixels por imagem             | 784 (28×28)       |
| valores por pixel             | 256               |
| contexto do modelo do Cap. 11 | <b>128 tokens</b> |

Passar pixel por pixel daria **784 tokens para uma única imagem** — seis vezes o contexto inteiro do nosso modelo. E seria desperdício: pixels vizinhos são quase sempre parecidos.

É exatamente o problema que o [Capítulo 6](#) resolveu para texto. Tokenizar byte a byte funciona e é ruim; o BPE aprendeu pedaços maiores e recorrentes.

**O VQ-VAE é o BPE das imagens.**

## 2. A ideia: aprender um alfabeto de pedaços de imagem

```
imagem --[encoder]--> vetores contínuos --[quantização]--> INTEIROS
                                         |
imagem' <--[decoder]-- vetores do codebook <-----+----->
```

O **encoder** é uma pilha de convoluções que reduz 28×28 para um mapa de 7×7, onde cada posição é um vetor de 32 números.

A peça nova é a **quantização**. Existe um *codebook*: K vetores aprendidos. Cada vetor que sai do encoder é substituído pelo **mais próximo** do codebook — e o índice desse vetor é o token.

```
dist = (plano.pow(2).sum(1, keepdim=True)
        - 2 * plano @ self.codebook.weight.t()
        + self.codebook.weight.pow(2).sum(1))
indices = dist.argmin(1) # <- o token
```

O resultado, medido:

| Medida     | Valor              |
|------------|--------------------|
| pixels     | 784                |
| tokens     | 49                 |
| compressão | 16x menos posições |

E uma imagem vira literalmente isto:

```
[24, 72, 64, 64, 24, 24, 72, 64, 72, 33, 71, 35, 35, 72, 64, 28, 28, 119, 119,
11, 72, 64, 43, 106, 71, 17, 52, 72, 64, 64, 35, 19, 11, 64, 72, 64, 64, 101,
92, 3, 64, 72, 95, 43, 77, 4, 64, 24, 78]
```

Uma lista de inteiros. Indistinguível, para o Transformer, de uma frase tokenizada.

### 3. O truque que faz isso treinar

`argmin` não tem derivada. Escolher o vizinho mais próximo é uma operação discreta — o gradiente morre ali, e o encoder nunca aprenderia.

A solução tem nome (*straight-through estimator*) e cabe numa linha:

```
z_q = z + (z_q - z).detach()
```

No **forward**, isso vale exatamente `z_q` — o vetor quantizado, que é o que o decoder deve receber. No **backward**, o termo `.detach()` não propaga nada, então o gradiente chega em `z` como se a quantização fosse a identidade.

É **aproximado**: fingimos que a quantização não existe na hora de calcular gradientes. E funciona bem o bastante para o encoder aprender a produzir vetores que quantizam bem.

#### As duas perdas

O codebook também precisa aprender. São duas parcelas, e as duas são necessárias:

```
perda_codebook = F.mse_loss(z_q, z.detach()) # puxa o codebook para o encoder
perda_commit = F.mse_loss(z, z_q.detach()) # puxa o encoder para o codebook
perda = perda_codebook + 0.25 * perda_commit
```

Sem a segunda — o *commitment loss* — o encoder foge livremente pelo espaço latente e o codebook nunca o alcança. Ela é o que obriga o encoder a **se comprometer** com os códigos que existem.

## 4. O resultado do VQ-VAE

1.500 passos, menos de dois minutos:

| Medida                           | Valor           |
|----------------------------------|-----------------|
| erro de reconstrução (validação) | 0,0039          |
| códigos usados                   | 93 de 128 (73%) |
| compressão                       | 16x             |

Original e reconstruída, em arte ASCII:

```
%@@@@@#####* : #@@@@@#*+*#***=
      : :::: @@= .:::-%@-
      @@- .%@-
      :@@: %@:
      %@: .@@.
      :@@: -@@:
      #@%. *@%.
      .%@= %@*
      :@@@: :@@@.
      =@% +%#.
```

Reconhecível, com as bordas mais grosseiras — é o preço de descrever a imagem com 49 símbolos de um alfabeto de 128.

### (!) 35 códigos nunca foram usados

Esse é o defeito clássico do VQ-VAE, e tem nome: **colapso do codebook**. Um código que nunca é o mais próximo de nada nunca recebe gradiente — e nunca se move para uma região onde seria útil. Ele morre no lugar onde foi inicializado.

É um problema de **realimentação**: para ser escolhido, o código precisa estar perto de alguma coisa; para chegar perto de alguma coisa, precisa ser escolhido. Quem começa mal fica mal.

Soluções conhecidas (o E4 mede uma): reinicializar códigos mortos sobre vetores do batch, usar médias móveis exponenciais em vez de gradiente, ou adicionar ruído no começo do treino.

## 5. A linha que é o capítulo inteiro

```
from modelo import GPT
```

É a **mesma classe** que escreveu prosa de Machado nos capítulos 11 a 15. Não há versão "visual" dela. Não há camada nova, nem atenção especial para imagens.

```
cfg = {"vocab_size": 128, "block_size": 49,  
      "n_embd": 128, "n_head": 4, "n_layer": 4}  
m = GPT(cfg)
```

O que mudou foi só o que entra:

| Capítulo    | Entrada | Tokenizador | Vocabulário | Tokens por amostra |
|-------------|---------|-------------|-------------|--------------------|
| Capítulo 11 | texto   | BPE         | 1.024       | 128                |
| Capítulo 17 | imagem  | VQ-VAE      | 128         | 49                 |

A arquitetura não sabe a diferença. Ela recebe inteiros e aprende a prever o próximo — e "o próximo" pode ser a próxima palavra ou o próximo pedaço de imagem.

**É por isso que modelos multimodais existem.** Uma vez que tudo vira token, o mesmo modelo serve. A parte difícil não é a arquitetura — é o tokenizador de cada modalidade.

## 6. O modelo gerando imagens

830.976 parâmetros, 2.000 passos, 4,4 minutos sobre 2,9 milhões de tokens:

| Medida            | Valor                |
|-------------------|----------------------|
| loss de validação | 1,3599               |
| perplexidade      | 3,9 entre 128 tokens |

Três imagens geradas do zero, token por token — nenhuma delas estava no dataset:

```
::: ...  
.,=* : =%%+ :%+  
: . %@@@@@%= . =+  
=@@@ :%@@@@@@@@@: *@@#  
-@@+ #@+ +@@# .#@%  
-@@@- #@= +@@@@@+ @@@.  
-@@#+*@@%: +@@@@@@@@: .@@#  
#@@@@@- :#@@@+ .@@%.  
.@@@@#=: . #@@* +@@=+@@@=  
::=++#@@%+ -@= :#@@% +@@@@@@%*+-
```

O processo é **idêntico** ao dos capítulos de texto: amostra um token, realimenta, repete 49 vezes. Depois o decoder do VQ-VAE transforma a sequência em pixels.

## E vale olhar com honestidade

Os traços parecem dígitos, e vários não são dígito nenhum. É o esperado: **0,8 M de parâmetros** treinados em **2,9 M de tokens** — a mesma conta desfavorável do [Capítulo 11](#).

Este capítulo demonstra o **mecanismo**, não a qualidade. Um modelo que gera imagens convincentes por este caminho existe (o DALL·E original fazia exatamente isto) e é três ordens de grandeza maior.

## 7. O que a prática faz diferente

O caminho deste capítulo — VQ-VAE mais Transformer autorregressivo — é real e foi o estado da arte por um tempo. O que mudou desde então:

| Abordagem                | Como difere                                                                                                            |
|--------------------------|------------------------------------------------------------------------------------------------------------------------|
| VQ-GAN                   | acrescenta uma loss adversária ao decoder; reconstruções muito mais nítidas                                            |
| Difusão                  | abandona os tokens discretos: aprende a remover ruído de forma iterativa, em espaço contínuo                           |
| ViT / encoders contínuos | para <i>entender</i> imagens (não gerar), pula a quantização e alimenta o Transformer com vetores contínuos de patches |

E há uma distinção que vale carregar: **gerar** e **entender** imagens pedem coisas diferentes.

Para gerar, você precisa de uma saída discreta que o modelo possa amostrar — daí a quantização. Para entender (responder perguntas sobre uma foto, por exemplo), basta projetar os patches direto no espaço de embeddings do modelo de linguagem, sem codebook nenhum. É o que fazem os modelos de visão-linguagem atuais.

## 8. Resumo do capítulo

- Um Transformer come **tokens**. Imagens são contínuas e longas — 784 pixels contra 128 de contexto.
- O **VQ-VAE é o BPE das imagens**: aprende um alfabeto de pedaços e comprime 784 pixels em 49 tokens.
- **argmin** não tem derivada; o **straight-through estimator** finge que a quantização é a identidade no backward. Aproximado, e funciona.
- O **commitment loss** é o que obriga o encoder a se comprometer com os códigos que existem.

- **Códigos mortos** são a falha clássica: 35 de 128 nunca foram usados. É um problema de realimentação — quem começa mal fica mal.
- `from modelo import GPT` — a mesma classe dos capítulos de texto, gerando imagens. Uma vez que tudo vira token, a arquitetura não sabe a diferença.

---

## Fim do curso

Você começou contando pares de letras no Capítulo 1 e terminou com o mesmo Transformer recebendo texto e imagem, servido por HTTP com batching.

Nada disso veio de uma biblioteca: o autograd, a atenção, o tokenizador, o otimizador, o KV-cache, a quantização e o VQ-VAE foram escritos aqui, peça por peça.

E o hábito que este curso tentou instalar não é nenhuma das técnicas — é o de **medir antes de concluir**. Os gabaritos guardam, ao lado das respostas certas, uma boa coleção de previsões minhas que a medição desmentiu. Elas estão lá de propósito.

# Exercícios — Capítulo 17 (Multimodal)

---

Faça na ordem. Soluções comentadas em [solucoes/](#) — **tente antes de olhar**.

Rode `python preparar_dados.py` e `python vqvae.py` antes. Juntos levam ~3 minutos.

---

## E1 — Por que quantizar (aquecimento)

Sem rodar:

1. Uma imagem 28×28 vira 49 tokens. Qual a taxa de compressão em **posições**? E se você contasse em **bits**, considerando que cada pixel é um byte e cada token cabe em 7 bits?
  2. Por que não alimentar o Transformer com os vetores contínuos do encoder, pulando a quantização? O que se perderia? (Dica: pense no que a camada de saída do modelo produz.)
  3. O capítulo chama o VQ-VAE de "BPE das imagens". Em que a analogia é boa, e em que ela quebra?
- 

## E2 — O straight-through estimator (importante)

1. Remova a linha `z_q = z + (z_q - z).detach()` e passe `z_q` direto para o decoder. Treine 300 passos. O que acontece com a perda de reconstrução, e por quê?
  2. Escreva o que o `.detach()` faz no forward e no backward, separadamente. Por que a expressão vale `z_q` num e `z` no outro?
  3. O estimador é **aproximado** — o gradiente que o encoder recebe não é o gradiente verdadeiro. Por que isso não impede o treino de funcionar?
- 

## E3 — As duas perdas (importante)

O quantizador tem `perda_codebook` e `perda_commit`.

1. Treine com `BETA_COMP = 0` (sem commitment loss). Meça o erro de reconstrução e o número de códigos usados. O que piora?
  2. Treine com `BETA_COMP = 2.0`. O que acontece agora, e por quê?
  3. Explique o papel de cada perda em uma frase, e diga o que aconteceria removendo a **outra** (a `perda_codebook`).
- 

## E4 — Códigos mortos

O capítulo mede 93 códigos usados de 128.



1. Varie o tamanho do codebook (`--codebook 32, 64, 256, 512`) e monte a tabela de códigos usados e erro de reconstrução. A fração usada cresce ou cai com o tamanho?
  2. Implemente a correção mais simples: a cada N passos, reinicialize os códigos não usados sobre vetores aleatórios do batch atual. Quantos códigos passam a ser usados?
  3. A correção melhorou o **erro de reconstrução**, ou só a estatística de uso? Se só a estatística, isso ainda vale a pena? Justifique.
- 

## E5 — O mesmo Transformer

1. No `gerar_imagens.py`, a config do GPT é `{vocab_size: 128, block_size: 49, ...}`. Compare com a do Capítulo 11. Quantas linhas de código do modelo precisaram mudar para ele aceitar imagens?
  2. A perplexidade medida é 3,9 entre 128 tokens. Compare com a do Capítulo 11 (51,7 entre 1.024). O modelo de imagens é "melhor"? O que torna a comparação difícil?
  3. O modelo gera as 49 posições em ordem de varredura (esquerda para a direita, de cima para baixo). Isso é natural para texto e arbitrário para imagem. Que problema essa ordem cria, e o que você tentaria no lugar?
- 

## E6 — Condicionar pelo dígito

O MNIST tem rótulos, e o `preparar_dados.py` já os salva.

1. Acrescente o rótulo como **primeiro token** da sequência (reserve 10 ids novos, como o Capítulo 14 fez com os tokens especiais). Retreine.
  2. Gere dez imagens, uma por dígito, começando cada sequência pelo rótulo correspondente. O modelo obedece?
  3. Você acabou de construir um gerador **condicional** — o mesmo mecanismo de um "texto -> imagem", com um vocabulário de condição de tamanho 10 em vez de uma frase. O que mudaria para condicionar por uma legenda de verdade?
- 

## E7 — Um modelo, duas modalidades (desafio)

1. Junte os vocabulários: tokens de texto do Capítulo 11 (ids 0-1023) e tokens de imagem deste capítulo (ids 1024-1151). Treine **um** GPT em sequências dos dois tipos.
2. Meça a loss em cada modalidade separadamente e compare com os modelos especializados. Houve interferência? Em qual direção?
3. **A pergunta que importa:** um modelo treinado nas duas modalidades separadamente, sem nenhum exemplo *pareado*, aprende alguma relação entre elas? Justifique pelo que o objetivo de treino recompensa — e diga o que seria preciso para que aprendesse.